

NoSQL Systems (DAS 838)

Assignment 1

Data Scaling, Data Integration and Unix Data Processing

Instructor: Vinu E. Venugopal

Team ID: 18, 28

Name	Roll number
Kalpit	BT2024093
Cheruku Sri Charan Reddy	BT2024143
Arnav Jain	BT2024233
D Sree Teja	BT2024160

15 September 2026

Contents

About this report	2
1 Scaling study with the TPC-H benchmark	2
1.1 What we set out to do	2
1.2 Experimental setup	2
1.3 How the runs were done	3
1.4 Results	4
1.5 What the numbers tell us	5
1.6 Limitations	7
2 Incomplete data integration and JSON generation	8
2.1 The problem and the data	8
2.2 Our matching strategy	8
2.3 Results	9
2.4 The JSON documents	9
2.5 Assumptions, limitations and difficulties	10
3 Unix data-processing pipeline	10
3.1 The task	10
3.2 The pipeline	11
3.3 Malformed records	11
3.4 Sample input and output	12
3.5 Scalability	12
3.6 Assumptions and limitations	13
4 Checklist against the assignment	13

About this report

This report covers all three problems of the assignment. For each one we describe what we built, how we tested it, what we measured and what we think the results mean. We have tried to back every number with a file in the submission, and to say so openly where something did not work out or where we could not find the cause of a result.

The submission folder mirrors the report: `problem1_tpch/`, `problem2_integration/` and `problem3_unix_pipeline/` each contain the code, the inputs, the raw outputs and a README with the same material in more detail. `COMPLIANCE.md` lists every requirement from the assignment PDF next to the file that addresses it; Section 4 gives a short version of it.

Machines. Problem 1 ran on a lab server with an Intel Core i9-12900K (16 cores, 24 threads), 62 GB of RAM and an NVMe SSD, under Red Hat Enterprise Linux 8.10. Problems 2 and 3 ran on a laptop with an AMD Ryzen 7 5800H (8 cores, 16 threads), 16 GB of RAM and an NVMe SSD, under Windows 11 with Git Bash (GNU awk 5.4.0, GNU coreutils 8.32) and Python 3.12.10.

1 Scaling study with the TPC-H benchmark

1.1 What we set out to do

The task was to run the TPC-H decision-support queries on a growing dataset, doubling the scale factor each time, and to work out how query time responds to the extra data. We used PostgreSQL, as the assignment suggests, and went from scale factor 1 up to 32.

We should mention that this is our second attempt. The first run used a copy of the TPC-H tools from GitHub (version 2.14) and generated the queries only once. That turned out to be a problem for Q11, whose threshold is supposed to shrink with the scale factor, so we threw those numbers away and repeated the whole study with the official kit and per-scale query generation. Everything below comes from the second run.

1.2 Experimental setup

Benchmark	Official TPC-H Tools v3.0.1 from tpc.org (7C5384AF-10DA-4DCB-B83F-BA2BDE51003F-TPC-H-Tool.zip), SHA-256 97ccb34cd122d78c2e06e2419e50957f934256868b37c02d0b88aefd9d13a84a
Tool banners	dbgen: “Version 3.0.0 build 0”, qgen: “v. 3.0.0 build 0” (this is what the programs in the 3.0.1 kit print)
Database	PostgreSQL 16.15, compiled from source, one build for all runs
Hardware	Intel Core i9-12900K, 62 GB RAM, 931.5 GB NVMe SSD (Crucial CT1000P3SSD8)
OS, compiler	Red Hat Enterprise Linux 8.10 (kernel 4.18), gcc 8.5.0
Scale factors	1, 2, 4, 8, 16, 32
Queries	all 22, generated separately for every scale factor with <code>qgen -r 42 -s <SF></code>
Repetitions	one warm-up run (not counted) and three measured runs per query
Timeout	30 minutes per query at every scale factor (never reached)

Table 1: Experimental setup for Problem 1.

The PostgreSQL settings were fixed before the first run and never touched afterwards. The database cluster was created with the C locale, which matters for the answer check described below.

```
shared_buffers = 4GB          effective_cache_size = 12GB
work_mem = 128MB             maintenance_work_mem = 1GB
max_parallel_workers_per_gather = 4
random_page_cost = 1.1       max_wal_size = 8GB
jit = off                    synchronous_commit = on
```

On top of the TPC-H schema we created primary keys on all eight tables, indexes on the foreign-key columns, and indexes on `l_shipdate` and `o_orderdate`. They are built after loading, the same way at every scale.

We ran SF1 to SF16 in one go and SF32 in a second session later the same day, because the disk on the server is shared and we had to free space in between. Both sessions recorded their environment in `results/environment.txt`. Apart from the date, the load average and the free memory and disk, the two records are identical. The load average of the machine (24 logical CPUs) was 3.74 when the first session started and 0.08 when the second one started.

1.3 How the runs were done

The whole procedure is in `run_benchmark.sh`. For every scale factor it does the following.

1. **Generate the queries.** `qgen` writes the 22 queries for that scale factor. We keep its exact output and a PostgreSQL version of each query, with SHA-256 checksums for both. The kit has no PostgreSQL output mode, so we built `qgen` for Oracle and changed two things afterwards: the row limit (`where rownum <= N`) becomes `LIMIT N`, and the `day (3)` precision in Q1's interval is dropped because PostgreSQL does not accept it. Scale-dependent parameters come straight from `qgen`; for example the Q11 threshold is 0.0001 at SF1 and 0.000003125 at SF32.
2. **Generate and load the data.** `dbgen -s <SF>` produces the eight tables, which are streamed into PostgreSQL with `COPY`. After loading, the script compares the number of rows in each table with the number of lines `dbgen` produced and stops if they differ. They matched for all 48 table and scale combinations, for example 6,001,215 `lineitem` rows at SF1 and 192,000,551 at SF32.
3. **Index and analyse.** The indexes are built and `VACUUM ANALYZE` is run. The load time and the database size are recorded.
4. **Check correctness (SF1 only).** The kit comes with official answers for SF1, computed with default query parameters. We generated that query set with `qgen -d`, ran it, and compared the results using the tolerances from the kit's own checker. All 22 queries passed, and the kit's checker script `cmpq.pl` reports zero unacceptable mismatches as well. The only value that is not identical is Q17 (official 348406.02, ours 348406.05). It is an average, for which the specification allows a difference of up to one percent.
5. **Save the plans.** The `EXPLAIN` output of every query is stored, so we can see later which join and scan methods PostgreSQL chose.
6. **Time the queries.** Each query runs once as a warm-up and then three more times. We record the wall-clock time around `psql` (connection and result transfer included) and whether the query succeeded.

All 528 query executions (6 scale factors $\times$ 22 queries $\times$ 4 runs) completed successfully. We use the median of the three measured runs.

1.4 Results

SF	Raw data (MB)	DB size (MB)	Load+index (s)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Median (s)	× prev.
1	1,049.7	1,704.4	18.4	6.23	6.16	6.31	6.23	–
2	2,114.6	3,397.7	38.1	16.41	15.08	13.84	15.08	2.42
4	4,254.5	6,783.8	83.2	36.48	38.15	37.05	37.05	2.46
8	8,555.8	13,555.6	155.4	69.48	63.63	60.50	63.63	1.72
16	17,220.6	27,102.6	308.4	121.70	123.07	122.74	122.74	1.93
32	34,688.3	54,197.3	636.1	354.27	316.77	328.23	328.23	2.67

Table 2: Dataset size, load time and total time of the 22 queries at each scale factor.

Query	Median time (s)						Ratio at each doubling					
	SF1	SF2	SF4	SF8	SF16	SF32	2/1	4/2	8/4	16/8	32/16	Slope
Q1	0.65	1.32	2.70	5.29	10.42	20.85	2.04	2.03	1.96	1.97	2.00	0.99
Q2	0.12	0.28	0.88	1.69	3.10	6.98	2.45	3.12	1.92	1.83	2.25	1.16
Q3	0.22	0.49	1.11	2.00	2.56	5.73	2.23	2.29	1.80	1.28	2.24	0.90
Q4	0.06	0.12	0.29	0.57	1.30	2.54	1.92	2.45	1.96	2.28	1.95	1.08
Q5	0.13	0.31	0.64	0.99	1.70	6.52	2.39	2.06	1.56	1.71	3.85	1.03
Q6	0.12	0.27	0.67	1.39	2.76	2.90	2.15	2.51	2.08	1.99	1.05	0.96
Q7	0.34	0.92	3.10	4.00	6.30	15.87	2.75	3.36	1.29	1.58	2.52	1.03
Q8	0.08	0.18	0.40	0.66	1.92	2.60	2.10	2.27	1.65	2.91	1.36	1.01
Q9	0.49	1.14	2.26	5.84	9.28	29.86	2.31	1.98	2.59	1.59	3.22	1.13
Q10	0.11	0.30	0.66	1.27	2.34	7.28	2.68	2.18	1.92	1.84	3.11	1.13
Q11	0.05	0.13	0.42	0.72	1.25	2.83	2.48	3.28	1.70	1.74	2.26	1.12
Q12	0.15	0.32	0.77	1.36	2.48	5.67	2.18	2.38	1.77	1.82	2.28	1.02
Q13	0.47	1.32	3.76	7.46	13.55	62.04	2.81	2.85	1.98	1.82	4.58	1.31
Q14	0.05	0.11	0.26	0.55	1.09	2.24	2.00	2.44	2.11	2.00	2.05	1.08
Q15	0.25	0.59	1.53	4.31	8.30	8.75	2.39	2.61	2.81	1.93	1.05	1.10
Q16	0.10	0.18	0.39	0.63	1.29	8.51	1.88	2.15	1.62	2.06	6.61	1.18
Q17	0.37	0.96	2.15	3.66	7.59	15.66	2.63	2.23	1.70	2.08	2.06	1.04
Q18	1.78	4.36	11.64	12.22	26.46	57.21	2.45	2.67	1.05	2.16	2.16	0.93
Q19	0.03	0.06	0.09	0.14	0.28	1.08	2.04	1.62	1.52	2.04	3.92	0.97
Q20	0.33	0.68	1.51	6.21	13.32	42.28	2.09	2.22	4.12	2.14	3.17	1.42
Q21	0.28	0.60	1.91	2.32	5.26	10.82	2.19	3.18	1.22	2.26	2.06	1.02
Q22	0.03	0.06	0.12	0.19	0.36	0.72	1.82	1.90	1.58	1.94	1.98	0.86

Table 3: Median time per query and the ratio between consecutive scale factors. A ratio of 2 means the time doubled along with the data. Ratios of 3 or more and of 1.4 or less are in bold. The slope is fitted to $\log(\text{time})$ against $\log(\text{dataset size})$ over all six scale factors, so 1 means linear growth.

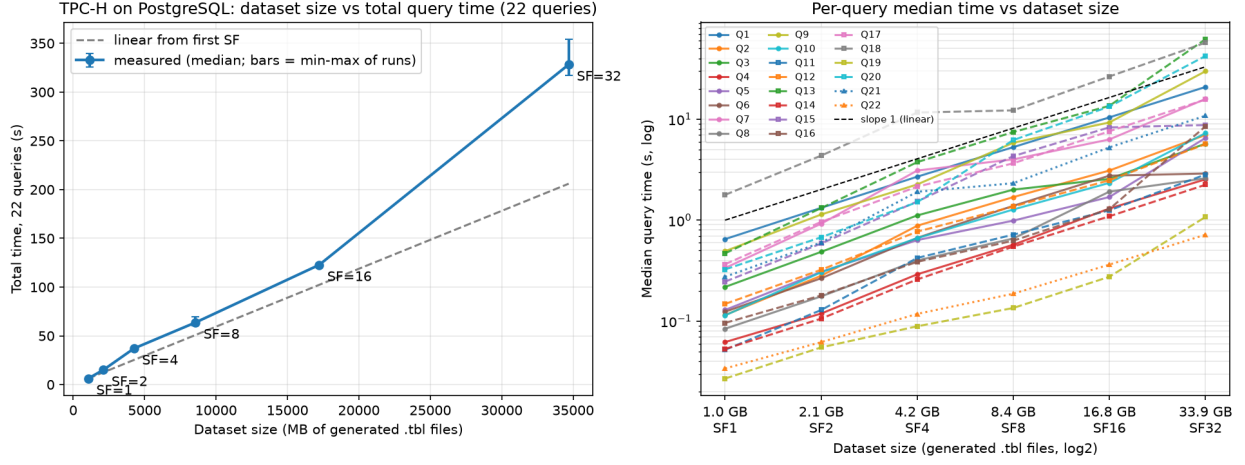


Figure 1: Left: total time of the 22 queries against dataset size, with the spread of the three runs and a line for purely linear growth from SF1. Right: median time of each query against dataset size on log-log axes.

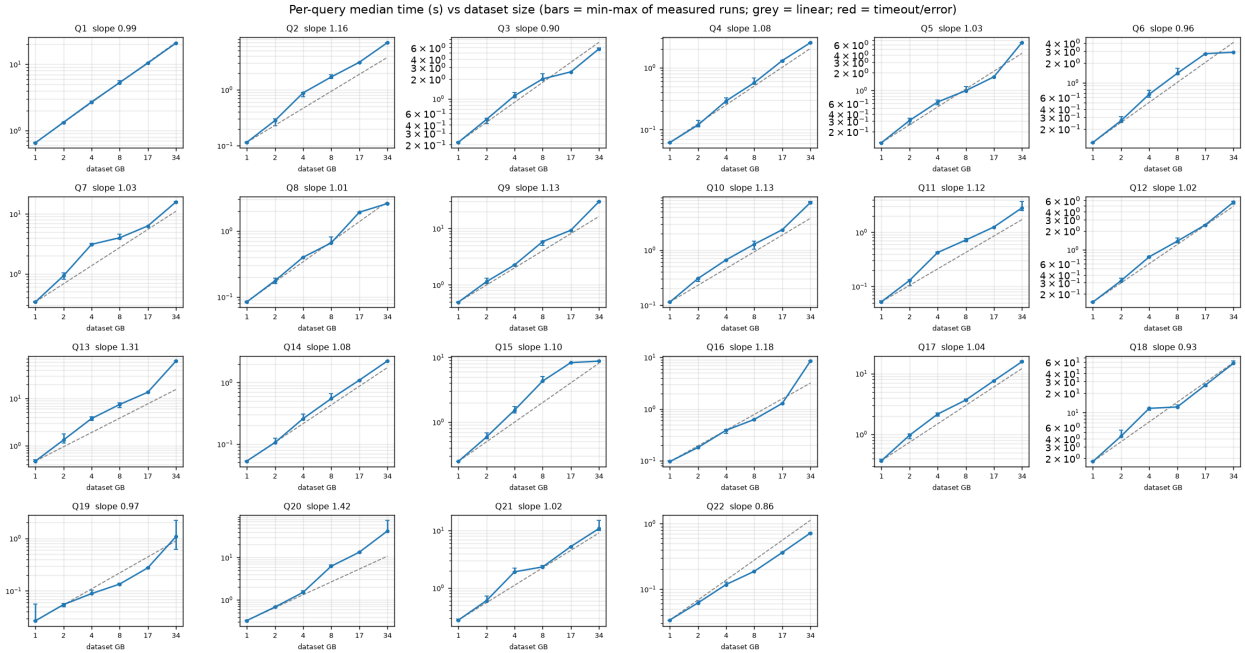


Figure 2: One panel per query. The grey line is linear growth from SF1; the bars show the minimum and maximum of the three measured runs.

1.5 What the numbers tell us

Does the time double when the data doubles?

Roughly, but not exactly, and not for every query. Between SF1 and SF32 the data grew by a factor of 33.0 and the total query time by a factor of 52.7, so the workload as a whole got slower a bit faster than the data grew. The ratio for the total stayed between 1.72 and 2.46 up to SF16 and then jumped to 2.67 at SF32, the highest of all the steps. Loading and indexing, on the other hand, scaled almost perfectly (ratios 2.07, 2.19, 1.87, 1.98 and 2.06), so the extra cost is not in reading more data but in how the queries are executed. Looking at the queries one by one, we found three different patterns.

Queries that scale linearly

Q1, Q4, Q12, Q14 and Q17 grow steadily: every ratio lies between 1.70 and 2.63, and their slopes are between 0.99 and 1.08. When we compared their saved plans, we noticed that PostgreSQL used the same plan shape for these queries at all six scale factors, which explains why their cost simply follows the number of rows.

Q1 is the clearest example, with ratios between 1.96 and 2.04. It filters `lineitem` on the ship date and groups the result into four groups. At SF32 the plan is a parallel sequential scan in five processes that returns 189 million of the 192 million rows; the aggregation needs only 24 kB of memory per process and the final sort 27 kB. The work is essentially reading every row once, so the time doubles with the data. Q12 and Q14 are similar filter-join-aggregate queries. Q17 is interesting because it contains a correlated subquery, yet it still scales linearly. Its plan does not change, which shows that a subquery on its own does not make a query super-linear.

Q22 grows a little more slowly than the data (slope 0.86). It is also the fastest query, at 0.03 s for SF1, and our timing includes starting `psql`, a fixed cost that does not depend on the data. For such a short query that fixed part is a noticeable share of the measurement.

Steps caused by the planner changing its mind

Several of the odd ratios in Table 3 line up exactly with a change in the saved plans between two scale factors.

Query, step	Ratio	Plan at the smaller scale → plan at the larger scale
Q6, SF16→32	1.05	bitmap scan on the <code>l_shipdate</code> index → parallel sequential scan
Q15, SF16→32	1.05	serial aggregation → parallel aggregation
Q18, SF4→8	1.05	serial hash aggregation → parallel partial hash aggregation
Q21, SF4→8	1.22	different join order and access paths
Q3, SF8→16	1.28	nested loops with index lookups → parallel hash joins
Q7, SF4→8	1.29	ordinary hash joins → parallel hash join with partial aggregation
Q8, SF16→32	1.36	more of the joins become parallel hash joins
Q20, SF4→8	4.12	hash semi-join → nested-loop semi-join running a subquery per row
Q5, SF16→32	3.85	index lookups into <code>lineitem</code> → parallel scan of <code>lineitem</code> and hash joins
Q10, SF16→32	3.11	index lookups into <code>lineitem</code> → parallel scan of <code>lineitem</code> and hash join

Table 4: Scaling steps that coincide with a plan change.

PostgreSQL picks plans by estimated cost. With small tables an index lookup is cheap; once many rows qualify, a sequential scan, a hash join or parallel workers become cheaper. When that switch pays off, the step looks almost flat (Q6, Q15, Q18, Q3). When the new plan does more work per row, the step is steep (Q20, Q5, Q10). So the flat steps do not mean a query is insensitive to the dataset size: they are one-off switches, and at the next doubling the time grows again (Q18 goes from 1.05 to 2.16, Q7 from 1.29 to 1.58 to 2.52, Q3 from 1.28 to 2.24). None of the 22 queries is insensitive to the data size over the whole range.

Where performance starts to degrade

The clearest degradation happens at SF32. The total grows by 2.67 there, and seven queries become more than three times slower in that one step: Q16 (6.61), Q13 (4.58), Q19 (3.92), Q5 (3.85), Q9 (3.22), Q20 (3.17) and Q10 (3.11). At this scale the database takes 54.2 GB, close to the 62 GB of RAM on the machine, while PostgreSQL’s own cache is set to 4 GB. To see where the time goes, we ran `EXPLAIN (ANALYZE, BUFFERS)` for these queries on the SF32 database.

- **Q13** counts orders per customer with a left join. At SF16 PostgreSQL used a hash join over a sequential scan of `orders`. At SF32 it switched to a merge join that reads `orders` through the customer index, which means reading the table in customer order instead of in the order it is

stored on disk. That index scan alone took 59.2 of the 65.2 seconds, with 18.4 million page reads outside PostgreSQL’s cache.

- **Q16** joins **part** with **partsupp** and counts distinct suppliers. At SF16 the join ran in parallel. At SF32 the join is serial, and sorting its result no longer fits into the 128 MB of **work_mem**, so PostgreSQL falls back to an external merge sort that writes 193 MB to temporary files.
- **Q9** is a six-table join. At SF32 its plan looks up **lineitem** through an index 1,390,984 times and **orders** by primary key 10,433,108 times, and each of its five parallel sorts spills about 135 MB to disk.
- **Q5 and Q10** switched from index lookups into **lineitem** to a parallel scan of the whole table joined against a large hash table built from the other join input (350 MB for Q5, 936 MB for Q10). Q10’s five parallel sorts also write about 138 MB each to disk.
- **Q20** contains a correlated subquery. Since its plan change at SF8, the subquery runs once for every candidate row of **partsupp**. At SF32 that is 786,335 separate index lookups into **lineitem**. It has the highest slope of all queries (1.42), and its three runs at SF32 took 75.7, 41.1 and 42.3 seconds, which fits a query that depends heavily on what happens to be in the cache.
- **Q18** groups **lineitem** by order. It scales linearly from SF8 onwards, but at SF32 the hash table no longer fits in memory: the final aggregation is split into 133 batches that use 4.0 GB of temporary disk, and each parallel worker writes about 1.3 GB more. With 57.2s it is the second slowest query at SF32, after Q13.
- **Q19** kept the same plan but still grew by 3.92. Its times are very small (0.28 s to 1.08 s) and its three runs at SF32 were 2.21, 1.08 and 0.61 seconds. With that much spread we do not want to claim a cause.

Relating this to joins, aggregation, sorting and filtering

Operation	What we observed	Where
Filtering and scanning	Linear. Parallel sequential scans spread the work over five processes.	Q1, Q12, Q14; Q6 at SF32
Aggregation	Linear while the hash table fits in work_mem ; much slower once it has to spill to disk in batches.	Q1 (4 groups, 24 kB) against Q18 (133 batches, 4.0 GB on disk at SF32)
Sorting	Negligible for small results; external merge sorts on disk once the input is larger than work_mem .	Q16 (193 MB), Q9 and Q10 (around 135–138 MB per sort) at SF32
Joins	The planner moves between index nested loops, hash joins and merge joins as the tables grow. These switches cause both the flat steps and the sudden jumps.	Q3, Q5, Q7, Q10, Q13, Q20, Q21
Correlated subqueries	Cost is roughly outer rows times inner lookup; super-linear when the plan runs the subquery row by row.	Q20 (786,335 executions); Q17 stays linear

Table 5: The observations grouped by database operation.

1.6 Limitations

- This is not an audited TPC-H result. We only measure single-stream query times, we do not run the refresh functions or the throughput test, and we added indexes. That is what the assignment asks for, but the numbers should not be compared with published TPC-H results.

- The server is shared with other users. It was fairly quiet when both sessions started, but we cannot rule out that someone else used it during our runs. The spread between runs is visible in Table 2 and in Figure 2.
- Some jumps have no explanation in the plans. Q2 (3.12), Q11 (3.28) and Q21 (3.18) grew quickly from SF2 to SF4 with the same plan shape, and so did Q19 at SF32. We report them without a cause.
- We captured `EXPLAIN ANALYZE` only at SF32. For the smaller scales we have the plain `EXPLAIN` plans, which show the chosen operators but not the actual row counts or memory use.
- SF32 was the largest scale we could run. It needs about 54 GB for the database and up to 34 GB more for the raw files during loading, and the shared disk had around 95 GB free, so SF64 would not have fit.
- Even with a warm-up run, the first measured run is sometimes slower (16.41 s against 13.84 s at SF2). That is why we report the median.

2 Incomplete data integration and JSON generation

2.1 The problem and the data

We had to compare every record in `customer_incoming.csv` (2,000 rows) with `customer_master.csv` (3,000 rows) and put each incoming record into exactly one of five categories: complete match, partial match, incomplete match, new entity, or conflicting information. Records that are missing values or disagree with the master should not simply be thrown away but written out as JSON.

Before writing any matching rules we looked at the data itself, because the rules only make sense if they fit the data. A few things stood out:

- In the master file, email, phone number and the combination of name and city are all unique. Names alone are not: 23 names appear more than once.
- Every incoming id that starts with `C` exists in the master file, and none of the 400 ids starting with `N` does.
- 62 incoming rows are completely empty, 104 contain only a name and 27 contain only a city.
- 145 customer ids appear two or three times in the incoming file (297 rows in total).
- Some values look odd but are still usable: 39 names are written in lower case with extra spaces around them (for example " chaaya verma "), and two cities have a trailing space.

2.2 Our matching strategy

The program (`match_customers.py`) looks for a master record using the strongest identifier the incoming record has, and stops at the first rule that finds exactly one candidate:

1. **Customer id**, exact match. It is meant to be the key, and our check showed that the ids are consistent.
2. **Email**, case-insensitive. It is unique in the master file.
3. **Phone**, comparing digits only. Also unique, but phone numbers are easier to mistype than emails, so it comes after email.
4. **Name and city together**, case-insensitive. Used only when nothing stronger is available.

If a rule would match more than one master record, we do not guess; the record is marked as incomplete. That never happens with this data, because the attributes are unique, but it keeps the program safe on other inputs.

We deliberately do not match on the name alone. 100 of the 104 name-only records have a name that appears exactly once in the master file, so matching them would have been tempting. However, 8 of the 400 new customers share a name with an existing customer, which shows that a unique-looking name is not enough to say two records are the same person.

Once a candidate is found, every value present in the incoming record is compared with the master record (after trimming spaces and ignoring case):

- all present values agree and nothing is missing: **complete match**;
- all present values agree but something is missing: **partial match**;
- at least one present value disagrees: **conflicting information**. We check conflicts before missing values, because a wrong value is more important to flag than an absent one.

Without a candidate, a record that has an id, email, phone, or name and city is a **new entity**; anything with less information, including the empty rows, is an **incomplete match**. Every row goes through the same code, and no record is handled manually.

2.3 Results

Category	Records	Share	Found through
Complete match	400	20.0 %	customer id (400)
Partial match	657	32.9 %	customer id (450), name and city (91), email (77), phone (39)
Incomplete match	193	9.7 %	no reliable candidate
New entity	400	20.0 %	no candidate
Conflicting information	350	17.5 %	customer id (350)
Total	2,000	100 %	

Table 6: Classification of all incoming records.

We checked whether the conflicts might just be formatting differences, such as spaces or punctuation. They are not: for all 350 conflicting records the values still differ after removing case, spaces and punctuation, for example the email `ekalinga.narayanan48812@example.in` against `ikshita.swamy1165@example.in`. 167 records conflict on one field and 183 on two; by field, there are 124 conflicts on phone, 108 on name, 104 on address, 101 on city and 96 on email.

The 145 repeated ids are classified row by row. As a result, one id can end up in different categories; C02666, for instance, is conflicting in row 27, a complete match in row 284 and a partial match in row 599. We kept this as it is, since the task is to relate each incoming record to the master data, not to merge the incoming records with each other.

2.4 The JSON documents

The program writes 1,202 JSON files: one for each of the 1,200 partial, incomplete and conflicting records, and two more for records whose only issue is an irregular value (one complete match and one new entity). A document for a conflicting record looks like this (a few short fields are put on one line here to save space):

```
{
  "source_row": 14,
  "customer_id": "C01165",
  "matched_master_id": "C01165",
  "name": "Ikshita Swamy",
  "available_information": {
    "name": "Ikshita Swamy", "phone": "8014527723",
    "address": "H.No. 977 Morar", "city": "Udupi"
  }
},
```

```

"missing_information": [],
"match_status": "conflicting",
"conflicting_information": {
  "email": { "incoming": "ekalinga.narayanan48812@example.in",
              "master": "ikshita.swamy1165@example.in" }
}
}

```

Records with irregular values carry an extra `irregular_information` block that keeps the raw value exactly as it arrived, together with what is wrong with it (for example “leading/trailing whitespace”).

Why JSON. In a CSV file every row has the same columns, so there is no clean way to say “this field is missing” as opposed to “this field disagrees with the master” or “this value was received with extra spaces” without inventing special placeholder values that are easily confused with real data. In JSON each record can simply carry the parts that apply to it: the available values, a list of missing fields, the conflicting values side by side, and the irregular raw values. Records with nothing missing do not need empty columns, and a reviewer can see at a glance what needs attention.

2.5 Assumptions, limitations and difficulties

- The matching is rule-based, not fuzzy. A misspelled name without a matching id, email or phone will be treated as a new customer. We accepted this on purpose: creating a duplicate customer is easier to fix later than merging two different people.
- Irregularity detection covers what we actually found in the data: surrounding spaces and names written entirely in lower case. We also checked that every phone has 10 digits, every email contains one @ and a domain, and every id has the form C or N followed by five digits. Misspellings are not detected.
- A record with both a missing field and a conflicting field fits two definitions. We had to pick one and chose conflicting, since that is the case a person should look at.
- The hardest part was deciding what “reliably matched” should mean without any ground truth. We could not measure accuracy against known answers, so we based the rule order on properties we could measure in the data, mainly how unique each attribute is.
- An empty row technically does not match any master record, which sounds like a new entity. We classify it as incomplete instead, because creating a customer with no information at all makes little sense.

3 Unix data-processing pipeline

3.1 The task

The goal was to compute the following query on a large tab-separated file using only Unix tools, without a database and without loading the whole file into memory:

```

SELECT category, COUNT(*) AS transactions, SUM(quantity * price) AS revenue
FROM transactions
WHERE date >= '2026-01-01' AND quantity > 2
GROUP BY category
HAVING SUM(quantity * price) > 100000
ORDER BY revenue DESC
LIMIT 10;

```

3.2 The pipeline

`pipeline.sh` takes the input file as an argument and runs the stages below, connected by pipes:

```
tail -n +2 | awk (validate, filter, project) | sort -k1,1
           | awk (group, aggregate, HAVING) | sort -k3,3gr | head -n 10
```

Stage	Unix command	SQL part
0	<code>head -n 1, tr, grep -nx</code>	find the column positions from the header (FROM)
1	<code>tail -n +2</code>	skip the header and stream the rows
2	<code>awk</code>	reject malformed rows into a side file (not part of the SQL)
3	same <code>awk</code> : <code>date >= "2026-01-01" && qty > 2</code>	WHERE
4	same <code>awk</code> : print category, quantity, price	projection
5	<code>sort -t\$'\t' -k1,1</code> with <code>LC_ALL=C</code>	bring each category together for GROUP BY
6	<code>awk</code> : count and sum while the category stays the same	GROUP BY, COUNT(*), SUM
7	same <code>awk</code> : keep a group only if revenue > 100000	HAVING
8	<code>sort -t\$'\t' -k3,3gr</code>	ORDER BY revenue DESC
9	<code>head -n 10</code>	LIMIT 10

Table 7: How each part of the SQL query maps to a Unix command.

A few design decisions are worth explaining. The columns are found by name, because the sample file supplied with the assignment (`customer_id product_id category quantity price date`) does not use the column order written in the PDF (`transaction_id date category quantity price`), while the generator does. The grouping `awk` only keeps the current category’s count and sum, since `sort` has already put equal categories next to each other; `sort` itself writes temporary files once its 256 MB buffer is full. No stage ever holds the whole file in memory. Revenue is printed with two decimals. An earlier version printed large sums in scientific notation (`7.2e+08`), which the final `sort` then read as 7.2 and ordered wrongly; we only noticed because Electronics and Furniture, the two largest categories, were missing from the output.

3.3 Malformed records

The validation `awk` checks every row and writes rejected rows, with the reason, to a separate file. The pipeline itself keeps running.

Check	Reason written
number of fields differs from the header (missing or extra field)	<code>field_count_<n></code>
empty category	<code>empty_category</code>
quantity is not a non-negative integer	<code>bad_quantity</code>
price is not a non-negative decimal number	<code>bad_price</code>
date is not YYYY-MM-DD or not a real calendar date	<code>bad_date</code>

Table 8: Validation rules.

Checking the format of the date alone was not enough. The generator’s malformed mode produces dates such as 2026-99-99, which look fine to a simple pattern and are even later than 2026-01-01, so they would end up in the result. We therefore also check the month and the number of days in the month, including leap years. The generator also produces rows with an extra field, which is why we check for a different number of fields rather than only for missing ones.

We tested this on the supplied `transactions_malformed.tsv` (four rejected rows) and on a generated file of 200,000 records with the `--malformed` option. In the latter, 1,907 rows were rejected: 386 bad dates, 382 bad prices, 390 bad quantities, 357 rows with a missing field and 392 with an extra one. The saved outputs are in `evidence/`.

3.4 Sample input and output

For the supplied `transactions.tsv` (20 rows), the pipeline prints:

category	transactions	revenue
Furniture	4	436000.00
Electronics	5	336000.00

Grocery (42,600) and Clothing (96,500) are removed by the `HAVING` condition. For `transactions_malformed.tsv` the result is Electronics (3 transactions, 178,000.00) and Furniture (1, 120,000.00), computed from the ten valid rows.

To be sure the pipeline is right, we also wrote a short reference implementation in Python (`reference_check.py`, used only for testing). Its output matched the pipeline exactly on the sample file, the malformed sample, the 200,000-row malformed file and the 100 MB generated file.

3.5 Scalability

We generated four datasets with `generate_transactions --records N --seed 42` and ran the pipeline three times on each, always on the same laptop and with the same settings. The file was read once beforehand, so every run started with it in the operating system’s cache.

Records	Size (MB)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Median (s)	s per MB
2,500,000	102.3	10.85	10.65	10.93	10.9	0.106
6,250,000	255.8	27.38	27.04	28.05	27.4	0.107
12,500,000	511.6	64.21	58.93	54.97	58.9	0.115
25,000,000	1023.2	109.45	107.86	109.37	109.4	0.107

Table 9: Total pipeline time for four input sizes.

The time grows linearly with the input: doubling the file roughly doubles the time, and the cost stays at about 0.11 seconds per MB from 100 MB up to 1 GB. The 500 MB runs varied the most (55 to 64 seconds); we suspect other programs on the laptop, but we did not verify this.

Which stage takes the time

Instead of guessing, we measured it. `pipeline.sh` can stop after a given stage (`STOP_AFTER`), so we timed the pipeline up to each stage in turn. The difference between two rows is the time that stage adds.

Pipeline up to ...	100 MB	250 MB	500 MB	1 GB
reading the file (<code>tail</code>)	0.7	1.0	1.5	2.5
+ validate, filter, project (<code>awk</code>)	10.4	25.7	49.9	97.8
+ sort by category	10.6	26.6	50.5	108.5
+ aggregate and <code>HAVING</code> (<code>awk</code>)	11.1	27.7	53.4	105.3
full pipeline	10.9	27.5	55.9	110.4

Table 10: Median time in seconds up to each stage (three runs).

The validation and filtering step takes about 90 % of the total. It is the only stage that handles every single row with several checks. Reading the file costs very little, so the disk is not the bottleneck. `sort` adds less than a second up to 500 MB, because it only receives the roughly 45 % of rows that pass the filter, the lines are short, and byte-wise comparison with `LC_ALL=C` is fast. Aggregation, the final sort and `head` add almost nothing, since at most 20 category rows reach them. At 1 GB the individual runs of the later stages vary by about ± 5 seconds, so small differences between neighbouring rows (such as “+ sort” being higher than “+ aggregate”) are noise.

This measurement also showed us where to optimise. The date check originally ran for every row, although the generator only produces about 730 different dates. Caching the result per date made the pipeline 18–27 % faster (149.9 s to 109.4 s at 1 GB) with identical output.

3.6 Assumptions and limitations

- The input has a header row naming the columns, as the supplied files and the generator output do.
- “At most 10 rows” refers to data rows; the output additionally starts with a header line naming the required columns.
- Negative quantities or prices, and numbers in scientific notation, are treated as malformed.
- The assignment shows the generator being called as `./generate_transactions 1000000`, but the supplied generator requires `--records 1000000`. We used the supplied file unchanged.
- Every run is a full scan. There are no indexes or query planner, and the dominant `awk` stage uses a single core; splitting the file and running several `awk` processes in parallel would be possible, but that is manual work a database would do for us.
- Revenue is summed as a floating-point number, so very large totals can be off by a few cents.
- We did not test inputs larger than 1 GB, which the assignment does not require.

4 Checklist against the assignment

The full mapping from each requirement to the files is in `COMPLIANCE.md`. In short:

Requirement	Status
<i>General</i>	
All code, dataset sources, commands, configurations and outputs submitted	met
Hardware and software environment documented; assumptions stated	met
Supplied files used unchanged	met (byte-identical)
<i>Problem 1</i>	
Official TPC-H 3.0.1 tools	met (SHA-256 recorded)
Scale factor doubled each step; each query and the total timed; repeated runs	met (SF1–SF32, 528/528 runs)
Environment unchanged across scale factors	met (shared server noted)
Plot of dataset size against query time, with an explanation	met
Linear, super-linear and insensitive queries; point of degradation; link to operations	met
<i>Problem 2</i>	
Every incoming record classified automatically into one of five categories	met
Matching strategy described and justified	met
JSON for missing, irregular and conflicting records; why JSON	met (1,202 documents)
Summary statistics, assumptions, limitations, difficulties	met
<i>Problem 3</i>	
Only Unix tools, streaming, no database	met
All SQL operations mapped to Unix commands	met
Malformed records detected and excluded without stopping	met
At least four sizes around 100 MB to 1 GB, same machine	met
Stage analysis, sample input and output, limitations	met
Generator called as in the PDF example	used the supplied generator’s <code>--records</code> option

Table 11: Summary of the compliance checklist.